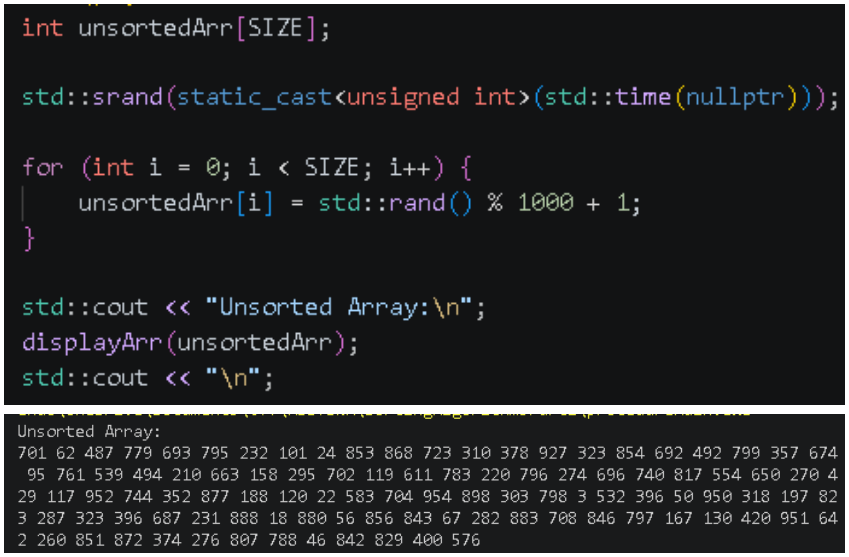
COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	September 9, 2026	**DATE SUBMITTED:**	September 15, 2026
NAME:	Mendoza, Nathaniel B.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 8 **Sorting Algorithms Part 2**

1. PROCEDURE OUTPUT

Code + Console Screenshot	
Observations	This prints 100 elements with random value from 1 to 1000 .

```
int unsortedArr[SIZE];

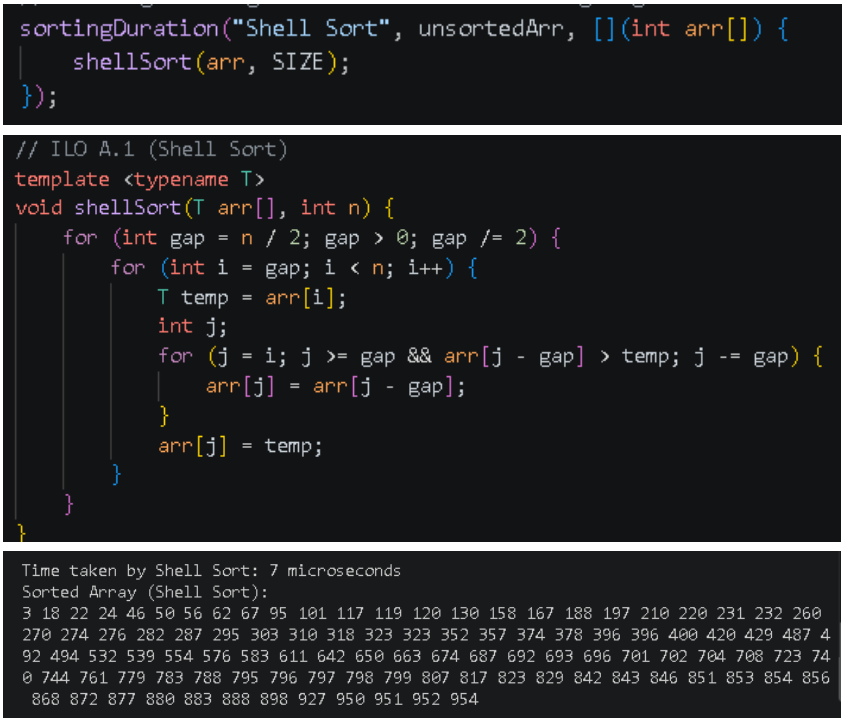
std::srand(static_cast<unsigned int>(std::time(nullptr)));

for (int i = 0; i < SIZE; i++) {
    unsortedArr[i] = std::rand() % 1000 + 1;
}

std::cout << "Unsorted Array:\n";
displayArr(unsortedArr);
std::cout << "\n";
```

```
Unsorted Array:
701 62 487 779 693 795 232 101 24 853 868 723 310 378 927 323 854 692 492 799 357 674
 95 761 539 494 210 663 158 295 702 119 611 783 220 796 274 696 740 817 554 650 270 4
29 117 952 744 352 877 188 120 22 583 704 954 898 303 798 3 532 396 50 950 318 197 82
3 287 323 396 687 231 888 18 880 56 856 843 67 282 883 708 846 797 167 130 420 951 64
2 260 851 872 374 276 807 788 46 842 829 400 576
```

Table 8-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	
Observations	Usage of Shell Sort Technique sorted this array by having increment gaps (n/2) which means its best case time complexity is O(N log N) because it swaps through gaps/distance which reduces the amount of shifting. Therefore, worst case time complexity of shell sort is $O(n^2)$.

```
sortingDuration("Shell Sort", unsortedArr, [](int arr[]) {
    shellSort(arr, SIZE);
});
```

```
// ILO A.1 (Shell Sort)
template <typename T>
void shellSort(T arr[], int n) {
    for (int gap = n / 2; gap > 0; gap /= 2) {
        for (int i = gap; i < n; i++) {
            T temp = arr[i];
            int j;
            for (j = i; j >= gap && arr[j - gap] > temp; j -= gap) {
                arr[j] = arr[j - gap];
            }
            arr[j] = temp;
        }
    }
}
```

```
Time taken by Shell Sort: 7 microseconds
Sorted Array (Shell Sort):
3 18 22 24 46 50 56 62 67 95 101 117 119 120 130 158 167 188 197 210 220 231 232 260
270 274 276 282 287 295 303 310 318 323 323 352 357 374 378 396 396 400 420 429 487 4
92 494 532 539 554 576 583 611 642 650 663 674 687 692 693 696 701 702 704 708 723 74
0 744 761 779 783 788 795 796 797 798 799 807 817 823 829 842 843 846 851 853 854 856
 868 872 877 880 883 888 898 927 950 951 952 954
```

Table 8-2. Shell Sort Technique

Code + Console Screenshot

```
sortingDuration("Merge Sort", unsortedArr, [](int arr[]) {
    mergeSort(arr, 0, SIZE - 1);
});
```

```
template <typename T>
void merge(T arr[], int left, int middle, int right) {
    int n1 = middle - left + 1;
    int n2 = right - middle;

    T *L = new T[n1];
    T *R = new T[n2];

    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];

    for (int j = 0; j < n2; j++)
        R[j] = arr[middle + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }

    delete[] L;
    delete[] R;
}
```

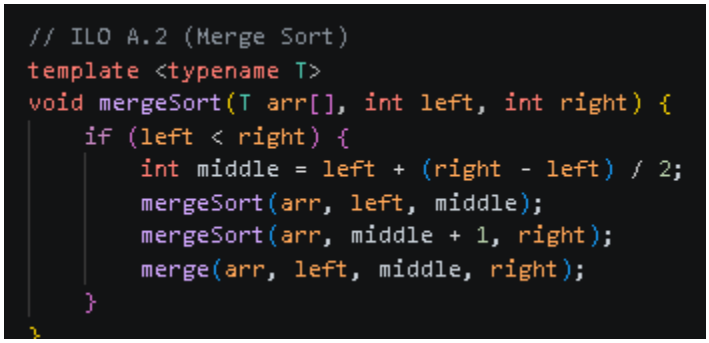
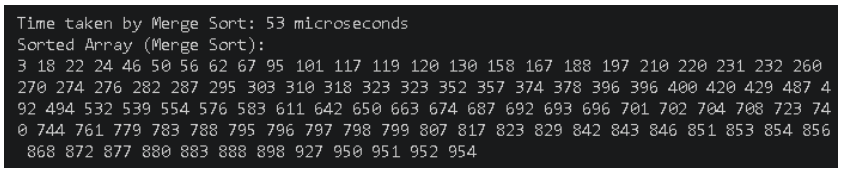
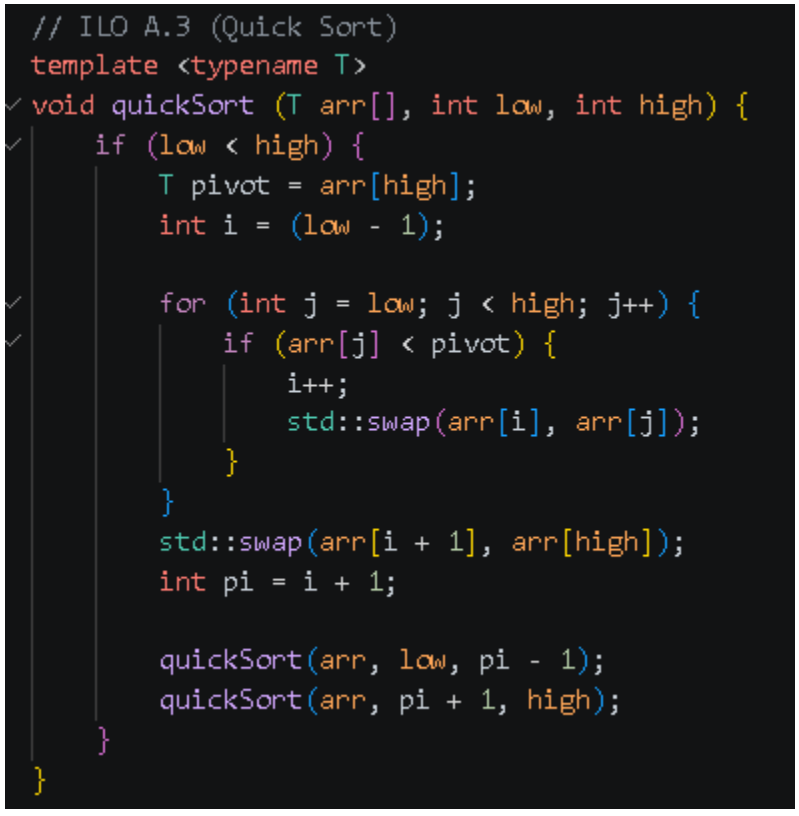
	 
Observations	This sorting technique always splits the array in half and takes the lowest number and merges it back to the original array.. Therefore its best and worst time complexity is O(N log N) because it has no input arrangements it always does split and merge and repeat.

Table 8-3. Merge Sort Technique

Code + Console Screenshot	

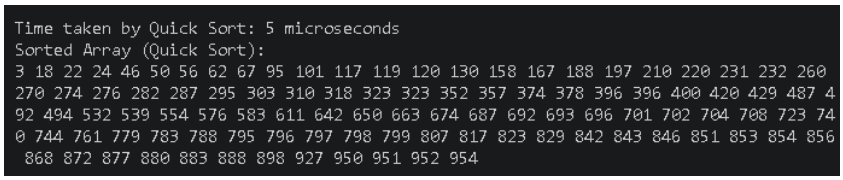
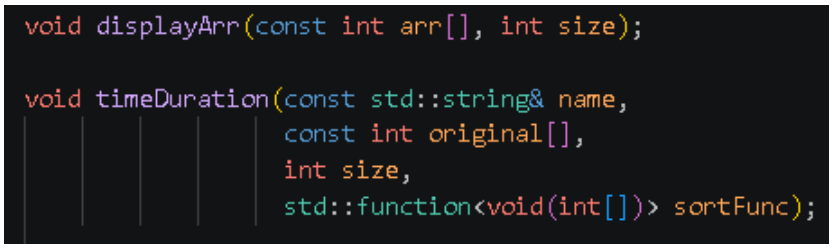
	 
Observations	Quick sort resulted the fastest among the other 2 sorting algorithms for this activity as its elements are swapped in-place with minimal memory overhead and fast cache reference locality. Wherein it partitions the array around the last element as a pivot and recursively places all smaller elements to its left. These shows that its best time complexity is O(N log N) and worst case is $O(N^2)$

Table 8-4. Quick Sort Technique

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Functions (SupplementaryMain.cpp)	

```cpp
void displayArr(const int arr[], int size) {
    for (int i = 0; i < size; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << "\n";
}

void timeDuration(const std::string& name,
                  const int original[],
                  int size,
                  std::function<void(int[])> sortFunc)
    {
    int tempArr[15];
    std::copy(original, original + size, tempArr);

    auto start = std::chrono::high_resolution_clock::now();
    sortFunc(tempArr);
    auto stop = std::chrono::high_resolution_clock::now();

    auto duration = std::chrono::duration_cast<std::chrono::microseconds>(stop - start);

    std::cout << name << " Time: " << duration.count() << " ns\n";
    std::cout << "Sorted: ";
    displayArr(tempArr, size);
    std::cout << "\n";
}
```

Header File Used
(For Supplementary)

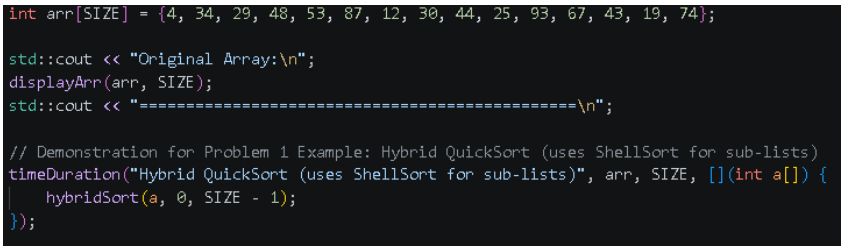
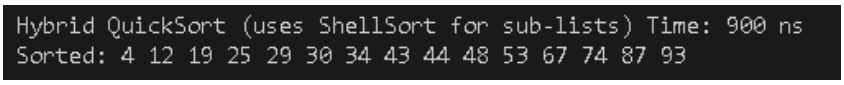
```cpp
// PROBLEM 1 EXAMPLE : Sorting left and right sub-lists using Shell Sort after partitioning with Quick Sort
void hybridSort(int arr[], int low, int high) {
    if (low < high) {
        // 1. Partition using Quick Sort logic
        int pivot = arr[high];
        int i = low - 1;

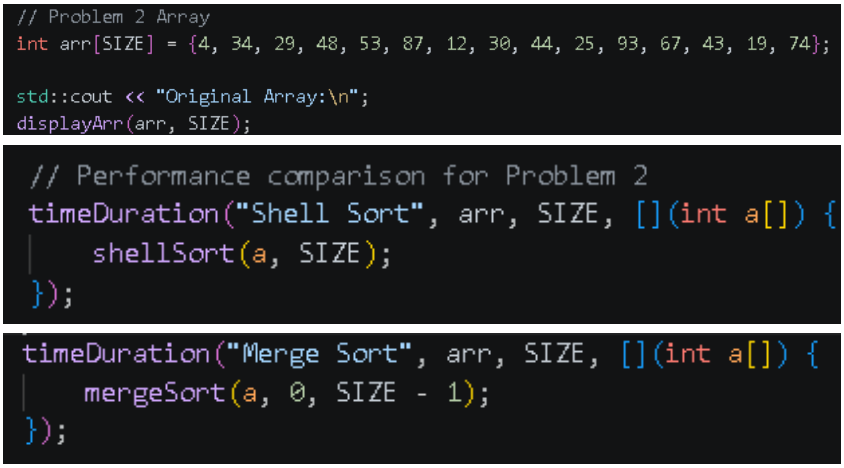
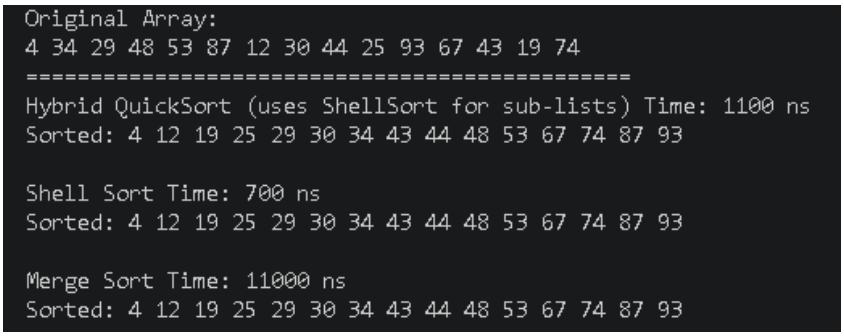
        for (int j = low; j < high; j++) {
            if (arr[j] < pivot) {
                i++;
                std::swap(arr[i], arr[j]);
            }
        }
        std::swap(arr[i + 1], arr[high]);
        int pi = i + 1; // Pivot position

        // 2. Sort left and right sub-lists using Shell Sort
        int leftSize = pi - low;
        int rightSize = high - pi;

        if (leftSize > 1)  shellSort(&arr[low], leftSize);
        if (rightSize > 1) shellSort(&arr[pi + 1], rightSize);
    }
}
```

```cpp
// Shell Sort to sort sub-lists (Qucik + Shell Sort) for Problem 1
void shellSort(int arr[], int n) {
    for (int gap = n / 2; gap > 0; gap /= 2) {
        for (int i = gap; i < n; i++) {
            int temp = arr[i];
            int j;
            for (j = i; j >= gap && arr[j - gap] > temp; j -= gap) {
                arr[j] = arr[j - gap];
            }
            arr[j] = temp;
        }
    }
}
```

Problem 1	
Code	
Output (With Duration / Time)	
Observations	This sorting technique is basically faster because Shell Sort handles the small sub-arrays instead of using more QuickSort steps. Therefore this hybrid quick sort algorithm (Combination of Shell and Quick Sort) sorted the array in 900 nanoseconds which shows the best time complexity is O(N log N) while worst case time complexity is $O(N^2)$.

Problem 2	
Code	
Output (With Duration / Time)	

Observations	Standard Shell Sort sorted the same 15-element array in 800 ns, making it slightly faster or to be specific 100 nanoseconds faster than the Hybrid QuickSort. Because the array is small, Shell sort alone is enough to perform these faster without combining sorting techniques.

What sorting algorithm will give the fastest time performance?	Shell Sort , in comparison to the time complexity to merge sort, is very fast compared to it.
Why can merge sort and quick sort have O(N log N) for their time complexity?	Merge Sort and Quick Sort can have O(N log N) because they split the data into smaller parts and then sort those parts efficiently. As the data gets bigger, the number of steps grows roughly with N(log N), instead of checking every possible pair.

Time Complexity Comparisons among the Hybrid, Merge, and Shell Sorting if the Array is big	
Code	

```
int main() {

    int arr[SIZE];

    std::srand(static_cast<unsigned int>(std::time(nullptr)));

    for (int i = 0; i < SIZE; i++) {
        arr[i] = std::rand() % 1000 + 1;
    }

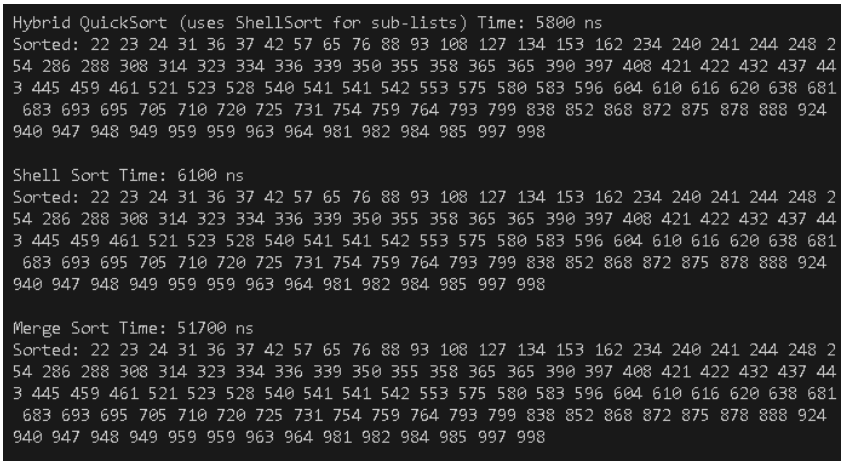
    std::cout << "Unsorted Array:\n";
    displayArr(arr, SIZE);
    std::cout << "\n";

    // Demonstration for Problem 1 Example: Hybrid QuickSort (uses ShellSort for sub-lists)
    timeDuration("Hybrid QuickSort (uses ShellSort for sub-lists)", arr, SIZE, [](int a[]) {
        hybridSort(a, 0, SIZE - 1);
    });

    // Performance comparison for Problem 2
    timeDuration("Shell Sort", arr, SIZE, [](int a[]) {
        shellSort(a, SIZE);
    });

    // Performance comparison for Problem 2 (Merge Sort)
    timeDuration("Merge Sort", arr, SIZE, [](int a[]) {
        mergeSort(a, 0, SIZE - 1);
    });

    return 0;
}
```

Output (With Duration / Time)	
Observations	This shows that there are big differences between merge, quick sort and the made hybrid sort which is the combination of Shell and Quick sorting technique. Where standard shell sort is slightly slower to the combined sorting algorithm.

CONCLUSION
Overall, I learned that Quick Sort, Merge Sort, and Shell Sort each have different ways of sorting data, and I used `chrono` to measure their running time and see which one was the fastest. Even though understanding the code behind each sorting algorithm can be difficult, testing and comparing their results helped me understand how they work better.

REFERENCES	
Sorting Algorithms	Shell Sort Program : Shell Sort - GeeksforGeeks
	Merge Sort Program : Merge Sort - GeeksforGeeks
	Quick Sort Program : Quick Sort - GeeksforGeeks
Chrono Usage for (TIme / Duration)	Internal Working of sort() in C++ - GeeksforGeeks c++ sorting algorithms duration - Stack Overflow

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic

work. The following indicates the nature and extent of AI assistance:

Tools Used: Google Search (Gemini?)

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: To find how to create the sorting algorithms in an approach to getting the duration.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: Learned how to apply such sorting algorithms while knowing how every line does and works, with the use of existing programs as references.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.